

# Course

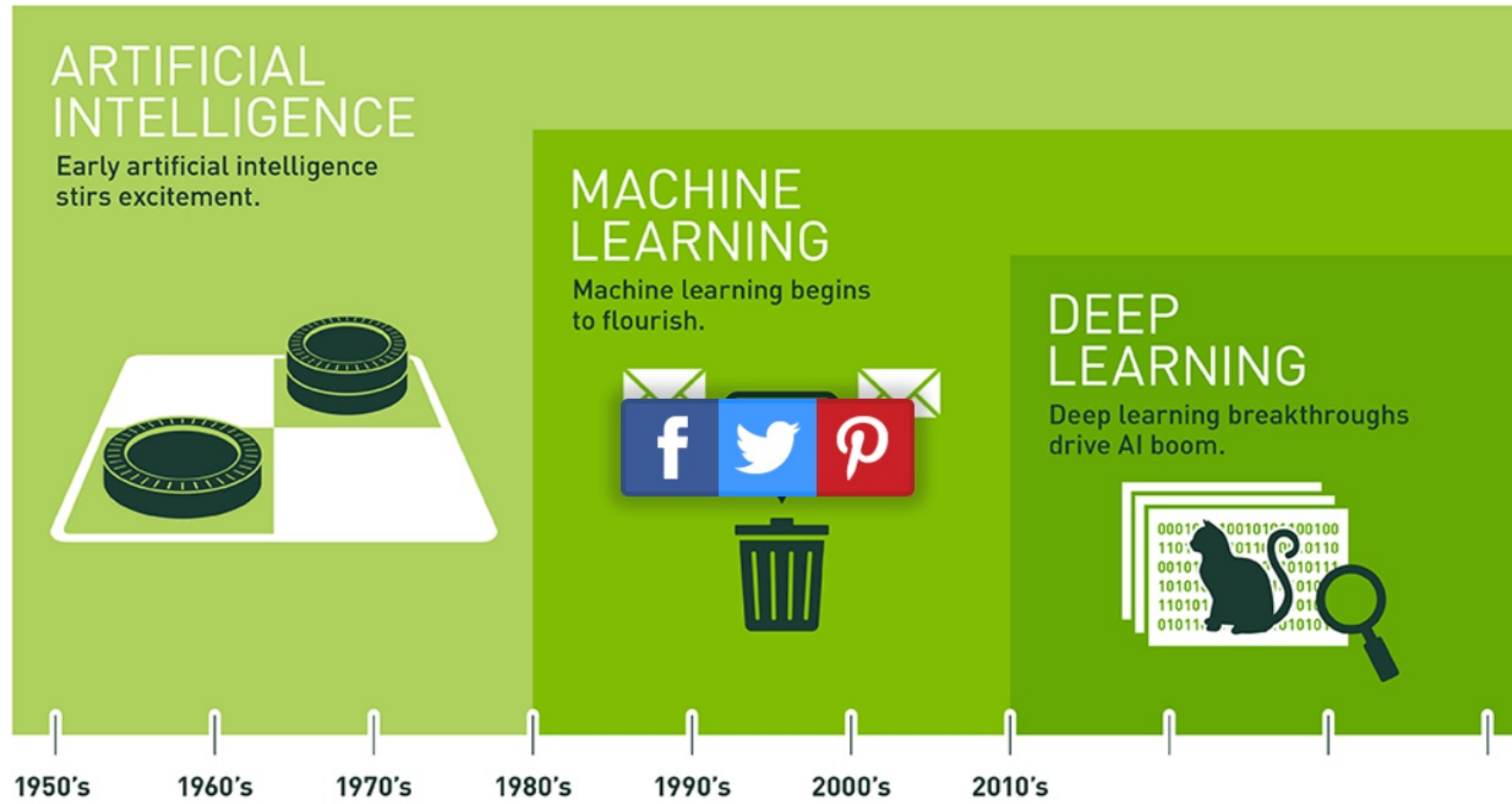
## « *Deep Learning* »

Sid-Ahmed Berrani

ENSIA

2024 – 2025

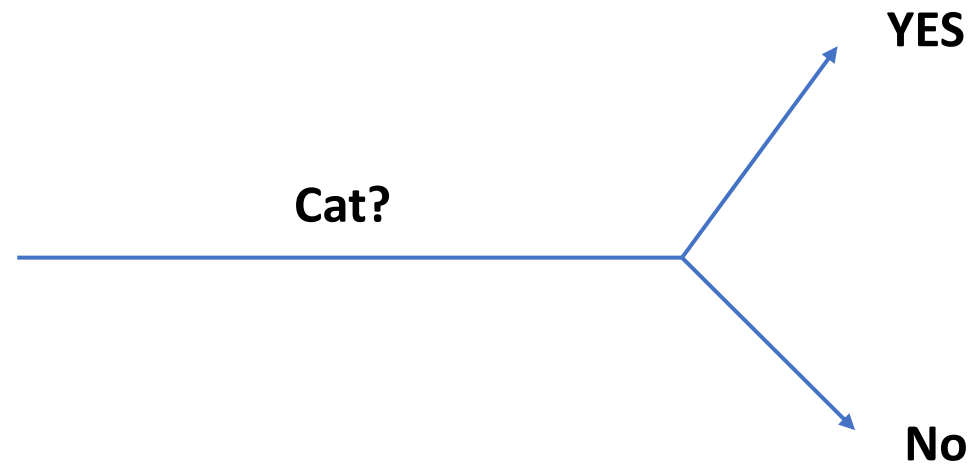
# Introduction



<https://blogs.nvidia.com/blog/2016/07/29/whats-difference-artificial-intelligence-machine-learning-deep-learning-ai/>

# Introduction

- Problem statement



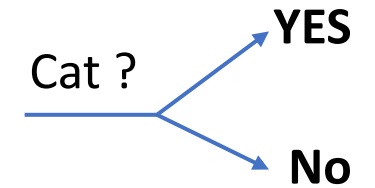
# Introduction

- Traditional Approaches

*We need an expert*

*The expert has to study the problem*

*A set of rules are derived (the dominant color, the shape, edges distribution...)*

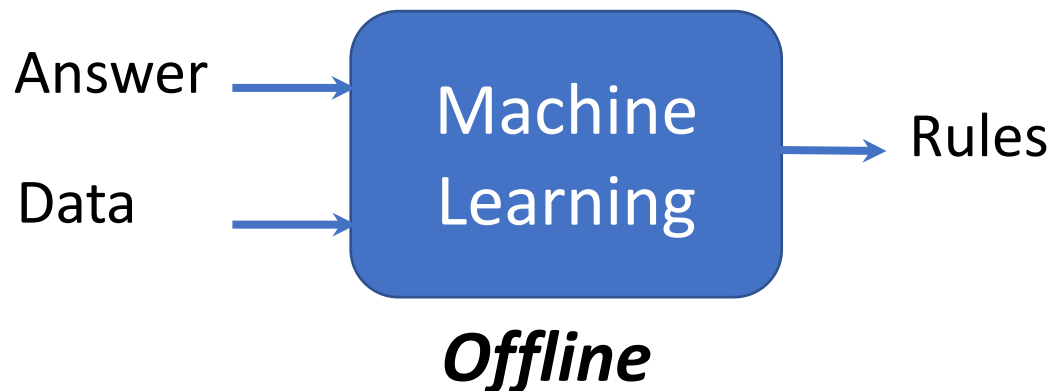
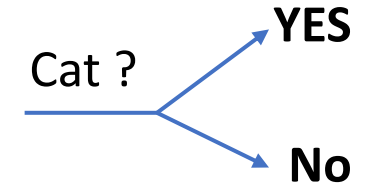


# Introduction

- Using machine learning: Two steps

**1<sup>st</sup> Step:** We use “data and answers” (examples) to automatically learn rules

**2<sup>nd</sup> Step:** Use learned rules in order to answer questions related to new “unseen” data

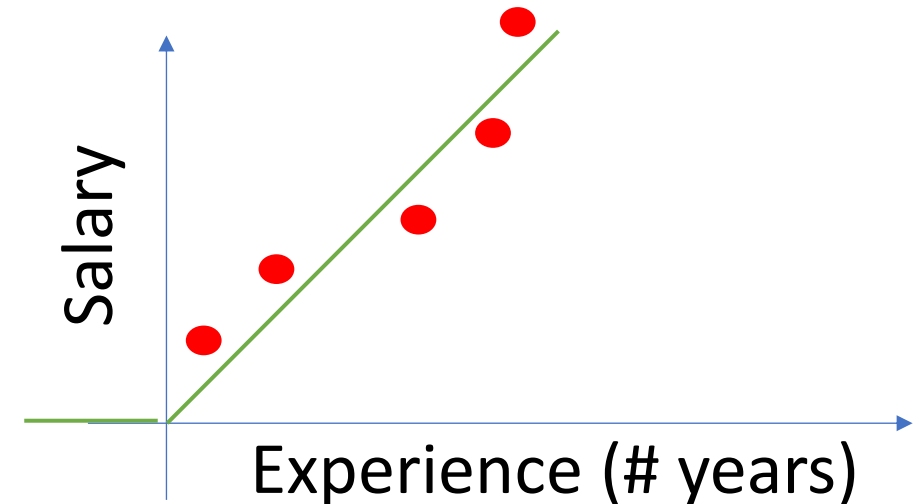
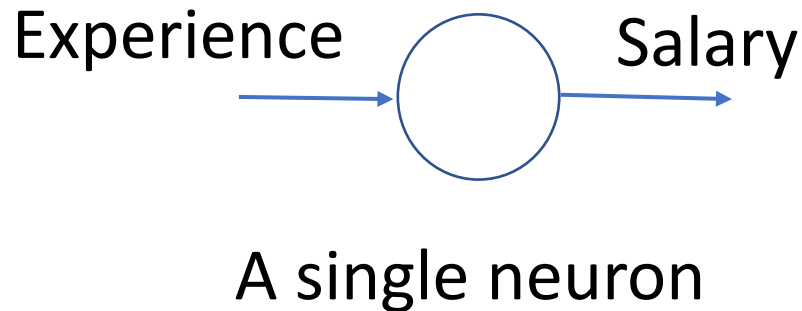


# Introduction

## Deep learning

- It refers to training neural networks.
- Deep: very large neural networks.

Let's consider the problem of salary estimation of data scientists.



# Introduction

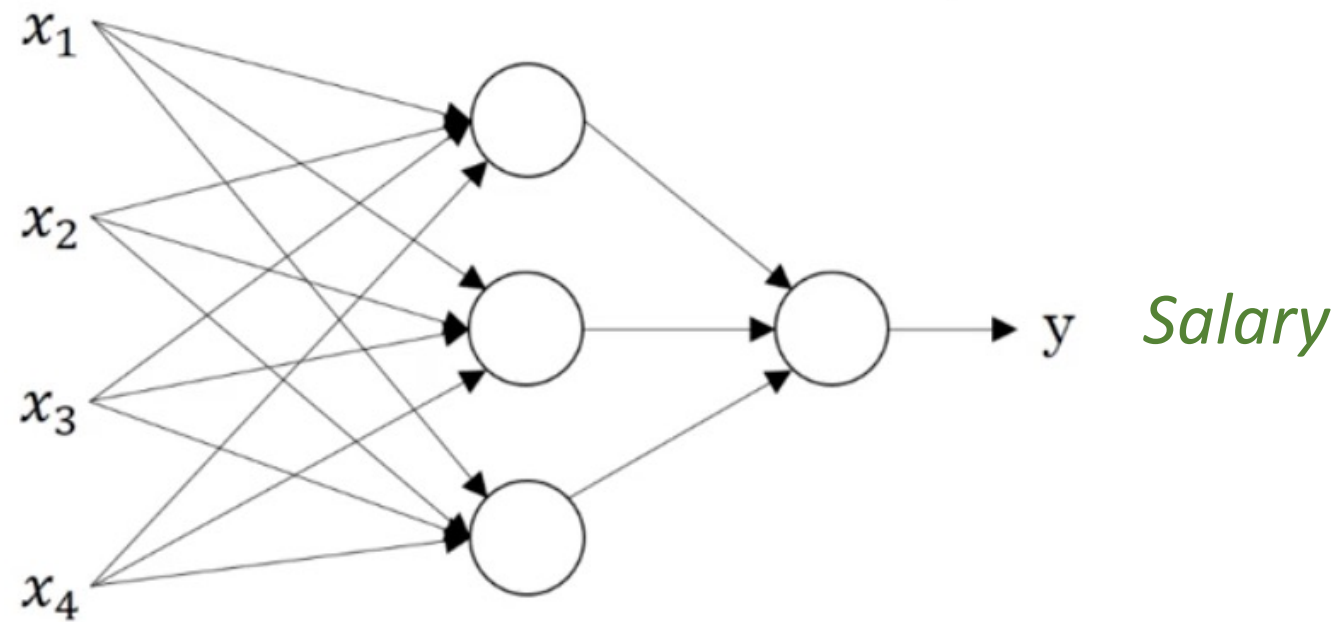
- We can add additional features:

*Experience*

*Region*

*NN expertise*

*Diploma*



# Introduction

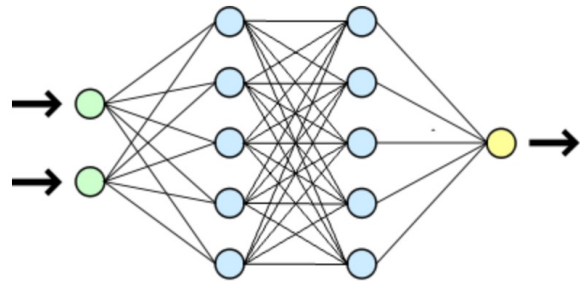
- Example used to tune the internal parameters of the neural network  
=> **Training data.**
- Perform training using labeled examples => **Supervised learning.**
- **Applications:**

|                     |    |                 |                        |
|---------------------|----|-----------------|------------------------|
| Medical exam (scan) | => | Diagnosis       | (Health)               |
| Purchased books     | => | next book       | (Recommandation)       |
| Image               | => | Object/class    | (Photo/video indexing) |
| Audio record        | => | Text transcript | (speech-to-text)       |
| English text        | => | French          | (Translation)          |

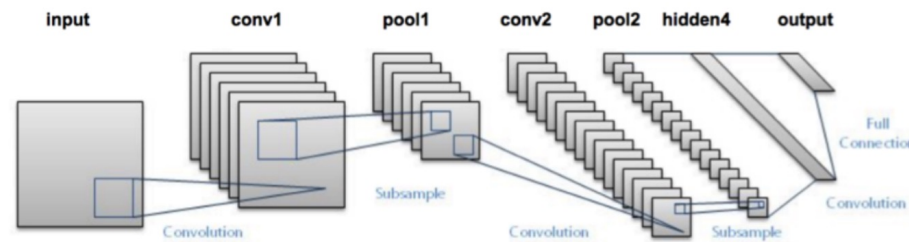


# Introduction

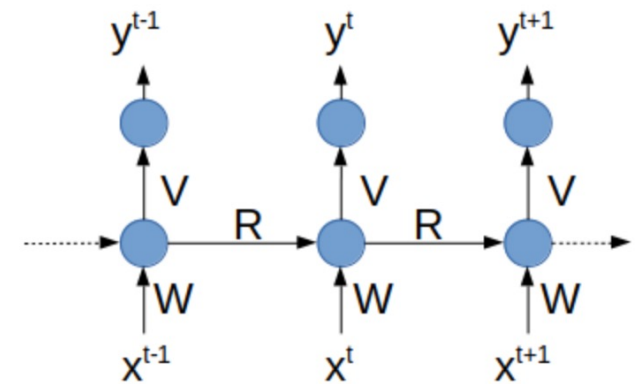
- Different types of NN architectures



Feed-forward NN  
(standard)



Convolutional NN



Recurrent NN

# Introduction

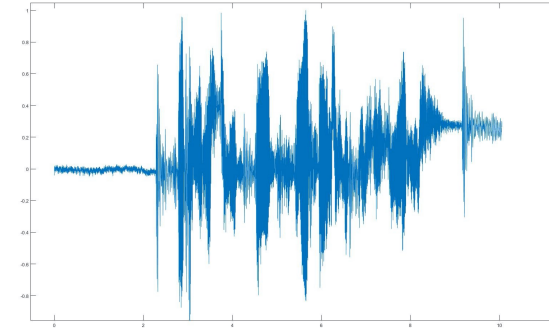
- **Two types of data**

## Structured data

| Experience | Expertise DNN | Age |
|------------|---------------|-----|
| 3          | 1             | 28  |
| 8          | 1             | 45  |
|            |               |     |
| 2          | 0             | 23  |

Each of the features has a very well defined meaning

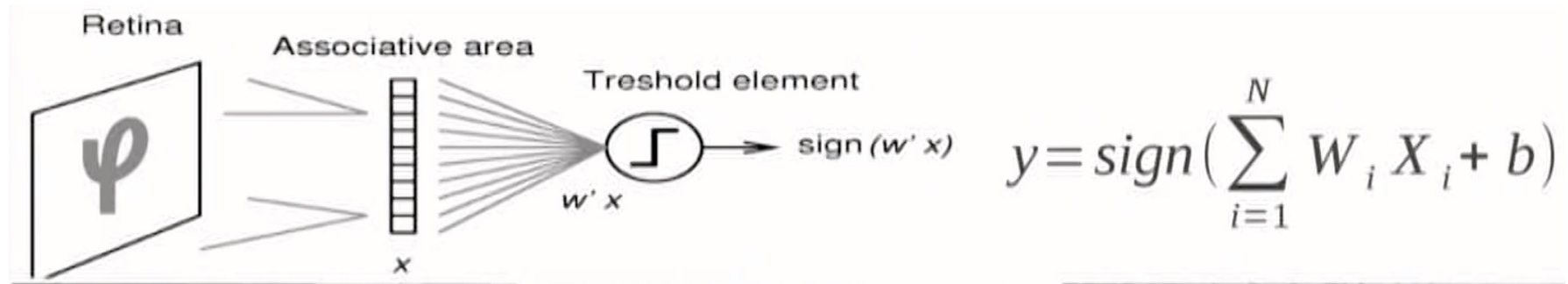
## Unstructured data



*Here are the most important criteria...*

# Introduction

- IA exists since very long time.
- Perceptron has been proposed in 1957 (Cornell)
  - A linear classifier.
  - Parameters are learned from examples.



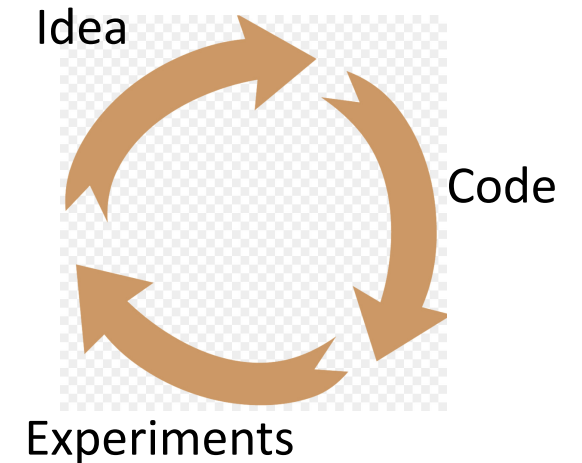
⇒ Why neural nets only just now taking off?

# Takeoff of Deep Learning: 4 Factors

1. Availability of sufficient amount of data for training.

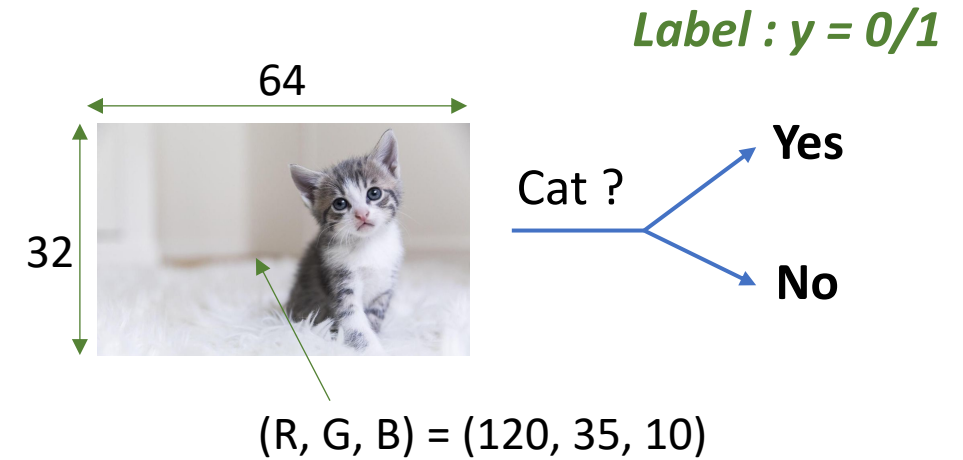
In particular, **labeled** data.

2. Availability of computation power for training.
3. Algorithms (very powerful and effective simple ideas).
4. Availability of training tools, platforms and trained models, released as *Open Source*.



# Neural Networks: Notation

- *Training examples =  $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$ .*
- *$X$  is a  $n_x$  by  $m$  dimensional matrix.*
- $X = [x^{(1)} x^{(2)} \dots x^{(m)}]$ .
- $X \in \mathbb{R}^{n_x \times m}$
- *$Y$  is a 1 by  $m$  dimensional matrix (vector).*
- $Y = [y^{(1)} y^{(2)} \dots y^{(m)}]$ .
- $Y \in \mathbb{R}^{1 \times m}$



↓

**A feature vector.  $x =$**

$$\begin{bmatrix} 120 \\ 45 \\ \vdots \\ 44 \\ 123 \\ \vdots \\ 209 \end{bmatrix}$$

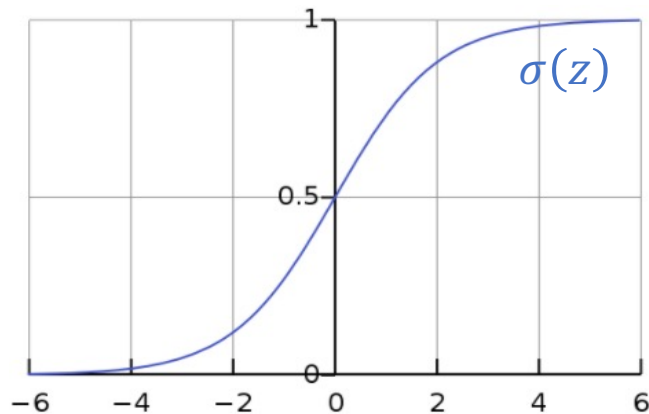
**Dimension :  $n_x = 32 \times 64 \times 3 = 6144$ .**  
 $x \in \mathbb{R}^{6144}$

# 1. Logistic Regression

## Binary classification Problem

- Given  $x$ , we want to estimate  $\hat{y} = P(y = 1|x)$ ;  $0 \leq \hat{y} \leq 1$ .
- $x \in \mathbb{R}^{n_x}$
- Parameters:  $w \in \mathbb{R}^{n_x}, b \in \mathbb{R}$ .
- Output:  $\hat{y} = w^T \cdot x + b$

$z$



$\sigma(z)$ : Sigmoid Function.

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$

If  $z$  is very large  $\Rightarrow \sigma(z) = \frac{1}{1+0} = 1$

If  $z$  is very large negative  $\Rightarrow \sigma(z) = \frac{1}{1+VL} \approx 0$

# 1. Logistic Regression

## The loss and the cost functions

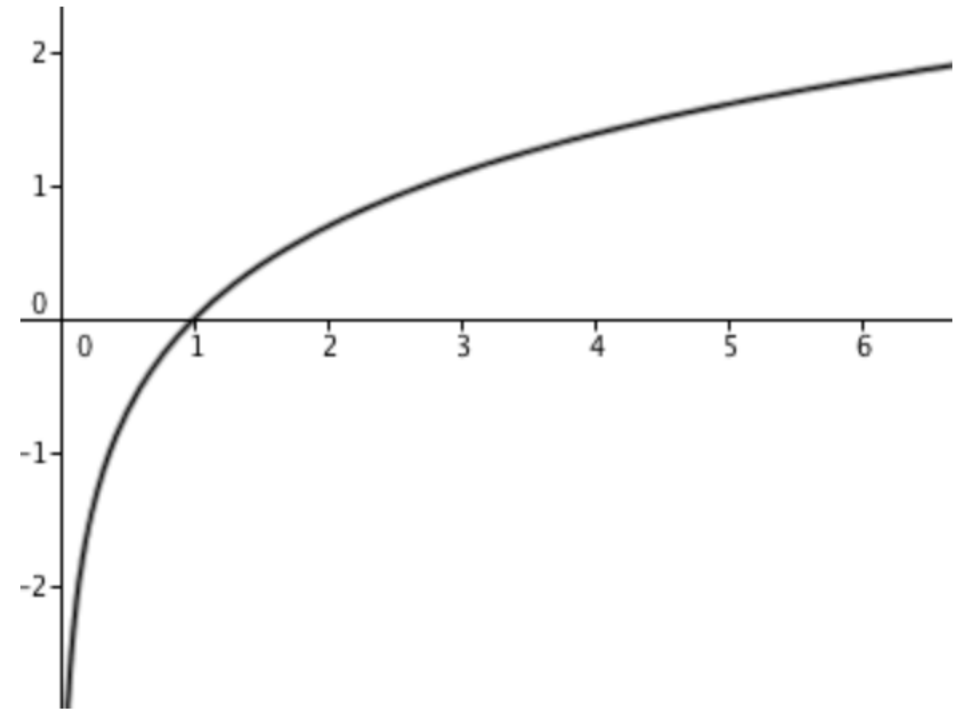
- To train the parameters  $w$  and  $b$  of the logistic regression model : We need to define a **cost function**.
- **Loss function** needs to be defined in order to **measure how good** the **obtained output** is with respect to the **expected output**.
- $\hat{y} = \sigma(w^T \cdot x + b)$ , where  $\sigma(z) = \frac{1}{1+e^{-z}}$
- Given  $\{(x^{(1)}, y^{(1)}), (x^{(2)}, y^{(2)}), \dots, (x^{(m)}, y^{(m)})\}$ , we seek:  $\hat{y}^{(i)} \approx y^{(i)}$ .
- **Loss (error) function:**  $\mathcal{L}(\hat{y}, y) = \frac{1}{2} (\hat{y} - y)^2$   
$$\mathcal{L}(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$
- **Cost function:**  $J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)})$ .

# 1. Logistic Regression

## The loss function

- Loss (error) function:  $\mathcal{L}(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$
- Why this loss function?
  - if  $y = 1$ :  $\mathcal{L}(\hat{y}, y) = -\log(\hat{y})$
  - if  $y = 0$ :  $\mathcal{L}(\hat{y}, y) = -\log(1 - \hat{y})$

*This loss function is called cross-entropy (also known as logarithmic loss or log loss).*





# 1. Linear Regression

## The cost function using MSE

Vector notation:

$$a(x) = w^T x$$

For a sample  $X$ :

$$a(X) = Xw$$

$$X = \begin{pmatrix} x_{11} & \dots & x_{1d} \\ \vdots & \ddots & \vdots \\ x_{\ell 1} & \dots & x_{\ell d} \end{pmatrix}$$

- Cost function:

$$J(w) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = \frac{1}{m} \sum_{i=1}^m (w^T \cdot x^{(i)} - y^{(i)}) = \frac{1}{m} \|Xw - Y\|^2$$

- Fitting the model to data:

$$J(w) \rightarrow \min(w)$$

- In this case,  $w$  can be found exactly and analytically:

$$w = (X^T X)^{-1} X^T Y$$

=> But we have to invert a matrix

=> Problem when  $d$  is large!

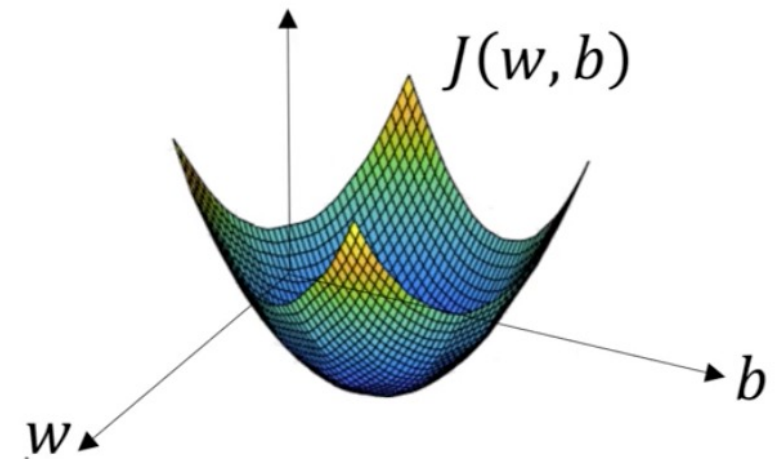
# 1. Logistic Regression

## Learning using gradient descent

- $\hat{y} = \sigma(w^T \cdot x + b)$ , where  $\sigma(z) = \frac{1}{1+e^{-z}}$

$$J(w, b) = \frac{1}{m} \sum_{i=1}^m \mathcal{L}(\hat{y}^{(i)}, y^{(i)}) = -\frac{1}{m} \sum_{i=1}^m y^{(i)} \log \hat{y}^{(i)} + (1 - y^{(i)}) \log(1 - \hat{y}^{(i)})$$

- **Objective:** find  $w$  and  $b$  that minimize  $J(w, b)$



# 1. Logistic Regression

## Learning using gradient descent

- How does it work?



*Repeat:*

$$w := w - \alpha \cdot \frac{dJ(w)}{dw}$$

$\alpha$  is the learning rate.

$$w := w - \alpha \cdot \frac{dJ(w, b)}{dw}$$

$$b := b - \alpha \cdot \frac{dJ(w, b)}{db}$$

# 1. Logistic Regression

## Learning using gradient descent

- We have:

$$\hat{y} = \sigma(w^T \cdot x + b), \text{ where } \sigma(z) = \frac{1}{1+e^{-z}}$$

$$\mathcal{L}(\hat{y}, y) = -[y \log(\hat{y}) + (1 - y) \log(1 - \hat{y})]$$

- To compute the gradients in this context, we need:
  1. Derivative of the sigmoid function:  $\frac{d\sigma(x)}{dx} = \sigma(x) \cdot (1 - \sigma(x))$
  2. Derivative of the log function:  $\frac{d \log(x)}{dx} = \frac{1}{x}$
  3. The chain rule.

# 1. Logistic Regression

## Learning using gradient descent

Derivative of the sigmoid function:

$$\frac{d\sigma(x)}{dx} = \sigma(x) \cdot (1 - \sigma(x))$$



Présentation

# 1. Logistic Regression

## Learning using gradient descent

### The chain rule

Let's explain it thru a simple example:

We know that:  $\frac{d \sin(x)}{dx} = \cos(x)$

But what about:  $\frac{d \sin(x^2+1)}{dx}$ ?

# 1. Logistic Regression

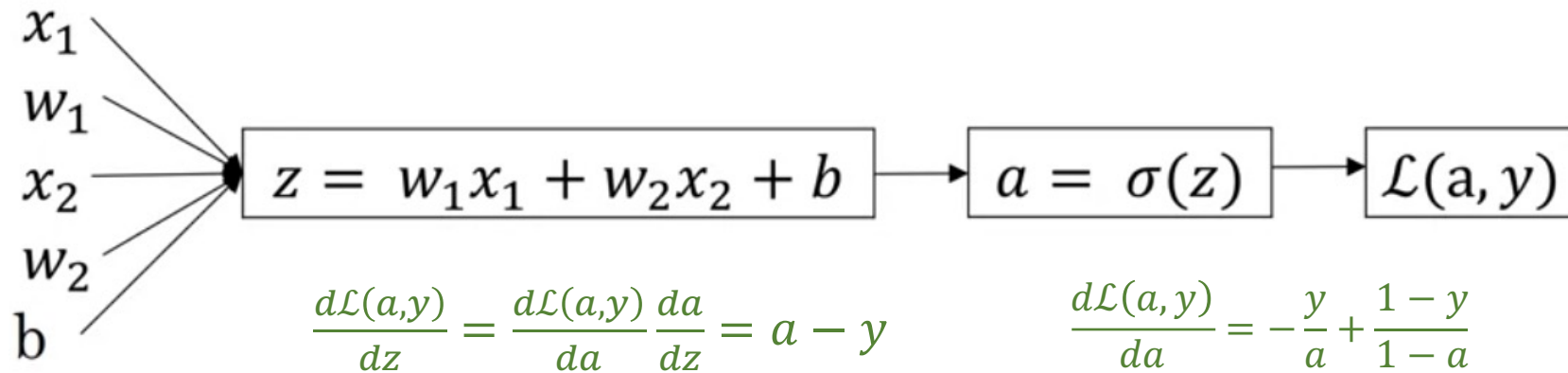
## Learning using gradient descent

- Application of gradient descent:

$$z = w^T x + b$$

$$\hat{y} = a = \sigma(z)$$

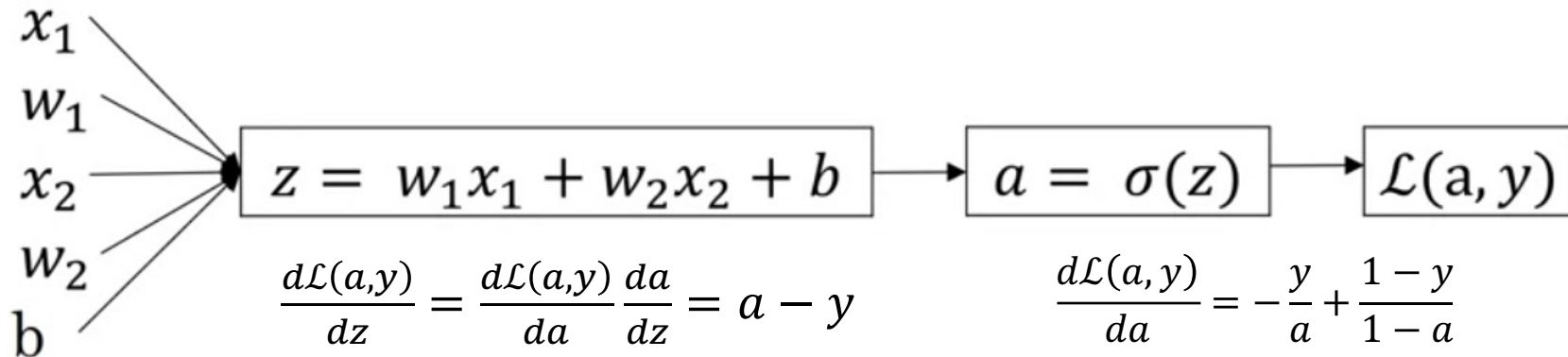
$$\mathcal{L}(a, y) = -(y \log(a) + (1 - y) \log(1 - a))$$



# 1. Logistic Regression

## Learning using gradient descent

- Application of gradient descent:



$$\frac{d\mathcal{L}(a, y)}{dw_1} = x_1 \frac{d\mathcal{L}(a, y)}{dz} = x_1(a - y)$$

$$\frac{d\mathcal{L}(a, y)}{dw_2} = x_2 \frac{d\mathcal{L}(a, y)}{dz} = x_2(a - y)$$

$$\frac{d\mathcal{L}(a, y)}{db} = \frac{d\mathcal{L}(a, y)}{dz} = a - y$$

$$w := w - \alpha \cdot \frac{dJ(w)}{dw}$$

$$w_1 := w_1 - \alpha \cdot x_1 \cdot (a - y)$$

$$w_2 := w_2 - \alpha \cdot x_2 \cdot (a - y)$$

$$b := b - \alpha \cdot (a - y)$$



# 1. Logistic Regression

## Learning using gradient descent

- Considering  $m$  examples:

$J = 0;$

$dw_1 = 0;$

$dw_2 = 0;$

$db = 0;$

**For  $i=1$  to  $m$ :**

compute  $z^{(i)}$

compute  $a^{(i)}$

compute  $\mathcal{L}(a^{(i)}, y^{(i)})$

$J = J + \mathcal{L}(a^{(i)}, y^{(i)})$

$dz = a^{(i)} - y^{(i)}$

$dw_1 += x_1^{(1)} * dz$

$dw_2 += x_2^{(1)} * dz$

$db += dz$

$J = J / m$

$dw_1 = dw_1 / m$

$dw_2 = dw_2 / m$

$db = db / m$

$w_1 := w_1 - \alpha . dw_1$

$w_2 := w_2 - \alpha . dw_2$

$b := b - \alpha . db$

# 1. Logistic Regression

## With another activation function: Tanh

$$\text{Tanh} : g(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}}$$

$$\text{Derivative of Tanh} : \frac{d g(z)}{dz} = 1 - \tanh^2(z)$$

### Required modifications:

- ...

# 1. Logistic Regression

## With another activation function: Tanh

- One downside of using tanh is that it can lead to slower convergence during training because the gradients can become very small when the outputs are near -1 or +1.
- Since the output is not constrained between 0 and 1, you must ensure that your interpretation of outputs aligns with the intended probabilistic interpretation in logistic regression.
- While it is technically possible to use a tanh activation function in a logistic regression context by mapping the target variable to -1 and 1, it deviates from the standard practice of using the sigmoid function.
- The sigmoid function is specifically designed for binary classification tasks. It provides a more natural probabilistic interpretation, making it the preferred choice for logistic regression.